

Allow Seeding Random Number Engines with `std::random_device`

Document number: P0205R0

Date: 2016-02-11

Project: Programming Language C++

Audience: Study Group 6 (Numerics), Library Evolution Working Group, Library Working Group

Reply-to: Moritz Klammler <moritz.klammler@gmail.com> (OpenPGP: 2732 DA32 C8D0 EEEC A081 BE9D CF6C 5166 F393 A9C0)

Table of Contents

- [1. Introduction](#)
- [2. Motivation and Scope](#)
 - [2.1 Background](#)
 - [2.2 The Problem with the Current Standard Library](#)
 - [2.3 The Proposed Solution](#)
- [3. Impact on the Standard](#)
 - [3.1 Clarification on Random Number Engine Requirements](#)
 - [3.2 Deprecation of Existing Features](#)
- [4. Design Decisions](#)
 - [4.1 Alternative Solutions](#)
 - [4.1.1 Provide a Generic Adapter Type instead of Modifying `std::random\_device`](#)
 - [4.1.2 Integer Types](#)
 - [4.2 Shortcomings](#)
 - [4.3 Impact on Standard Library Implementations](#)
 - [4.4 Implementations](#)
- [5. Proposed Wording](#)
- [Acknowledgments](#)
- [References](#)

1. Introduction

The purpose of this proposal is to make properly seeding a random number engine in a non-deterministic way easy and fast.

In order to achieve this, the following changes are proposed:

- Introduction of a new concept *seed generator*, which is a relaxation of *seed sequence* (§ 26.5.1.2 [rand.req.seedseq]).
- Addition of a member function `generate` to `std::random_device` in order to make it model the new concept.

The changes are non-breaking and can be implemented as a pure library solution with current C++14 features.

2. Motivation and Scope

2.1 Background

C++11 introduced a powerful random number generation library (§ 26.5 [rand]) under the `<random>` header. It provides *random number engines* (§ 26.5.1.4 [rand.req.eng]) that can be used either directly as a source of uniformly distributed pseudo random integers of unsigned type or together with *random number distributions* (§ 26.5.1.6 [rand.req.dist]) to produce pseudo random numbers according to a variety of distributions.

If an engine has an internal state of N bits, it can produce at most 2^N different sequences and the longest sequence it can produce will have a period of at most 2^N values until it necessarily starts repeating itself. Applications that wish to produce high-quality pseudo random numbers will therefore choose an engine such as the Mersenne twister with a large

internal state. Choosing an engine with a large internal state helps ensuring that the period of the generated sequence will be large. However, in order to ensure that the number of different sequences the engine can produce is also exponential in the size of its state, it has to be made sure that the initial state is evenly distributed across all possible states. This is done by *seeding* the random number engine. If each of the 2^N states should be chosen with equal probability as the initial state, seeding requires 2^N bits of entropy.

The standard library provides the type `std::random_device` (§ 26.5.6 [rand.device]), a *uniform random number generator* (§ 26.5.1.3 [rand.req.urng]) that is supposed (but, unfortunately, not required) to produce a non-deterministic sequence of uniformly distributed integers of type `unsigned int`. The natural choice for an application that wishes to generate pseudo random numbers and does not need (and often doesn't want) reproducibility is to use it for seeding a random engine that is then used to produce pseudo random numbers.

2.2 The Problem with the Current Standard Library

Unfortunately, the current standard library does not provide any convenient way to use a `std::random_device` to properly (in the sense that each initial state is equally likely) seed a random engine.

The naïve approach that most people seem to use is the following.

```
template <typename EngineT>
    // requires(RandomNumberEngine(EngineT))
void
seed_non_deterministically_1st(EngineT& engine)
{
    std::random_device rnddev {};
    engine.seed(rnddev());
}
```

This code is severely flawed. If `EngineT` is `std::mt19937`, it has a state size of 19 968 bits. However, if an `unsigned int` is 32 bits (as is the common case on many platforms today), then of the up to $2^{19\,968}$ states, at most 2^{32} (that is one $2^{-19\,936}$ -th) can possibly be chosen!

To illustrate that this is in fact a real problem, O'Neill [1] has pointed out that a `std::mt19937` engine seeded like above can never produce certain integers as its first value. This can have had consequences on real-world programs. A program that incorrectly assumes that an impossible number will eventually be produced as the engine's first (or n -th) output is broken in a very subtle way. After all, it would take extensive and practically unrealistic unit testing to do an exhaustive search over the possible seeds and even detect the bug.

In addition to seeding an engine with an integer, the standard library also provides a way to seed it with a so-called *seed sequence* (§ 26.5.1.2 [rand.req.seedseq]). A seed sequence may be constructed in a deterministic way from zero or more integers that provide some initial entropy. The numbers are then possibly scrambled and stored in some internal state of the seed sequence which can be externalized using the `param` member function. Such a memento may then be used to re-create a seed sequence of the same type in the same state. A seed sequence provides a member function `generate` that takes a pair of random access iterators and assigns a uniformly distributed unsigned 32 bit integer to each element in the range denoted by the iterator pair. The standard library provides a single implementation of a seed sequence in `std::seed_seq` (§ 26.5.7.1 [rand.util.seedseq]). This type can be used to seed a random engine more thoroughly.

```
template <typename EngineT, std::size_t StateSize = EngineT::state_size>
    // requires(RandomNumberEngine(EngineT))
void
seed_non_deterministically_2nd(EngineT& engine)
{
    using engine_type = typename EngineT::result_type;
    using device_type = std::random_device::result_type;
    using seedseq_type = std::seed_seq::result_type;
    constexpr auto bytes_needed = StateSize * sizeof(engine_type);
    constexpr auto numbers_needed = (sizeof(device_type) < sizeof(seedseq_type))
        ? (bytes_needed / sizeof(device_type))
        : (bytes_needed / sizeof(seedseq_type));
    std::array<device_type, numbers_needed> numbers {};
    std::random_device rnddev {};
    std::generate(std::begin(numbers), std::end(numbers), std::ref(rnddev));
    std::seed_seq seedseq(std::cbegin(numbers), std::cend(numbers));
    engine.seed(seedseq);
}
```

```
}
```

This code has a number of problems.

- It is absurdly complicated for what could rightfully be expected to be a simple task. (Even the author is not absolutely sure it is correct.) It is unrealistic (and unreasonable) to expect a casual user to come up with such a seeding procedure.
- It is not as general as one might hope it is. The *random number engine* concept does not require that an engine exposes its `state_size` as `std::mersenne_twister_engine` does. (The author believes that making this constant part of the requirements would have been a good idea but it is too late for this now.) This forces the user of the function to look up the actual state size in the documentation and provide the correct value as the second template parameter. (One could write type traits for the engines provided by the standard library and fall back to `sizeof` as a reasonable estimate for third-party types, faithfully assuming they won't allocate memory to hold state on the free store.)
- It is not as accurate as it should be. Assuming that `std::random_device` produces truly independent uniform random numbers, `std::seed_seq` actually *introduces* non-uniformity [1]. (O'Neill provides experimental evidence for this. Anyway, it is clear that a deterministic algorithm can only ever reduce but never increase the entropy of a given seed.)
- It is not as efficient as it could be. While all that's really needed is copying a sequence of random bytes into the internal state of the engine, the `std::random_device` first copies them into a stack-based `std::array` from which the `std::seed_seq` copies them into a heap allocated (!) internal buffer, scrambles them in a (in this case counter-productive) attempt to remove bias until the engine finally copies them to their final destination. To make matters worse, the `std::random_device` does not know in advance how many bytes will be needed. Therefore, on implementations where it reads from a file such as `/dev/urandom`, it cannot buffer the reads efficiently.

2.3 The Proposed Solution

With this proposal adopted, properly seeding a random engine could be as simple as this.

```
template <typename EngineT>
    // requires(RandomNumberEngine(EngineT))
void
seed_non_deterministically_3rd(EngineT& engine)
{
    std::random_device rnddev {};
    engine.seed(rnddev); // note: passing rnddev itself as opposed to the
                        // result of invoking its call operator
}
```

This would be made possible by adding a `generate` member function to `std::random_device` directly and relaxing the type requirements on the argument to the `seed` member function to only require that `generate` member function.

There would be no unnecessary copies, no unneeded tempering, no dynamic memory allocation, no introduced bias and little chance to get anything wrong on the user's side.

3. Impact on the Standard

This proposal could be implemented with very little changes, none of which would be breaking anything. No core language changes are needed.

The requirements on the type parameter for a random engine's `seed` member function (and the corresponding constructor) should be relaxed such that it only requires the `generate` member function from *seed sequence*. Bolas [2] has argued that even today, a conforming implementation isn't allowed to *call* anything but `generate` and perhaps `size` due to existing complexity requirements. Unfortunately, it may still enforce their presence via type checks.

To allow also other types than `std::random_device` to be used, a new concept *seed generator* that only specifies the `generate` member function should be introduced and the *seed sequence* concept be defined as a refinement of it.

The specification of `std::random_device` should be changed such that it meets the requirements of *seed generator*. The only thing that is needed here is the addition of the `generate` member function.

This proposal has no dependencies on any other proposal. The author is not aware of any other proposal that depends on

or conflicts with this proposal. In particular, it is orthogonal to N3547 [5]. (However, the sample implementation of `std::randomize` in that paper could benefit from the feature suggested in this proposal.)

3.1 Clarification on Random Number Engine Requirements

Currently, table 117 in section 26.5.1.4 [rand.req.eng] defines the various overloads of the seed member function of a type `E` that meets the requirements of *random number engine* in terms of the corresponding constructor and equality operator. For example, `e.seed(q)` is defined to have the effect that “post: `e == E(q)`” and complexity “same as `E(q)`”. This is unfortunate because for a *seed sequence* `q`, two successive calls to `q.generate` (even for ranges of identical size) do not have to produce the same sequence of numbers, even though `std::seed_seq` happens to behave that way. The requirements for *seed sequence* don't mandate this; the respective row in table 115 says about `q.generate(rb, re)` (emphasis added):

Does nothing if `rb == re`. Otherwise, fills the supplied sequence `[rb, re)` with 32-bit quantities that depend on the sequence supplied to the constructor **and possibly also depend on the history of `generate`'s previous invocations.**

Therefore, the author believes that it is not intended by the current standard that the following code should be guaranteed to work.

```
template <typename RandEngT, typename SeedSeqT>
    // requires(RandomNumberEngine(RandEngT) && SeedSequence(SeedSeqT))
void
test(RandEngT& engine, SeedSeqT& seedseq)
{
    engine.seed(seedseq);
    assert(engine == RandEngT {seedseq}); // might fire
}
```

Regardless of whether this proposal will be adopted, the wording in table 117 should be updated to make it clear that calling `seed` puts the engine into the same state as calling the corresponding constructor but not create the impression that the assertion in the above example would necessarily hold.

3.2 Deprecation of Existing Features

It is not proposed that any existing library features be deprecated.

Possible candidates for deprecation could be

- `std::random_device::operator()` and
- the overload of `E::seed` that takes a single value of type `E::result_type` and the corresponding constructor of a *random number engine* `E`.

The `operator()` of `std::random_device` should definitely stay and not be deprecated. `std::random_device` meets the requirements of *uniform random number generator* and as such needs that function. Its presence is also not the root cause of the problem this proposal sets out to solve. The problem is not how the integer is obtained but that using a single integer as a seed is insufficient.

Deprecating `E::seed` overloaded for `E::result_type` and the corresponding constructor would be more sensible because those functions actually encourage poorly seeding random number engines. On the other hand, they remain useful in situations where people want to hard-code a specific seed (such as in `std::mt19937 {42}`) and don't really care about uniformly choosing an initial state from the entire state-space of the engine.

4. Design Decisions

4.1 Alternative Solutions

The proposed solution falls into two parts:

- relaxing the type requirements on the seed member function and
- making `std::random_device` comply with these relaxed requirements.

The author is not aware of any substantially different alternative solution regarding the relaxation of the type

requirements. This is confirmed by the observation that O'Neill [1] has independently suggested the same change back in April 2015.

4.1.1 Provide a Generic Adapter Type instead of Modifying `std::random_device`

Instead of adding a member function to `std::random_device`, a new adapter type could be provided that would turn any *uniform random number generator* into a *seed generator*. This would have the advantage that one could also use, say, a `std::mt19937` engine to seed another `std::ranlux24` engine. (However useful that may be.) On the other hand, it would require more typing for the common case of seeding any random engine with a `std::random_device` as the temporary adapter object would have to be created. Such an adapter could also not take advantage of the size of the range being known ahead of time.

Anyway, here is a suggestion for this alternative.

```
template <typename UrngT>
    // requires(UniformRandomNumberGenerator(UrngT))
class seed_gen
{
private:
    UrngT urng_;

public:
    template <typename... ArgTs>
    seed_gen(ArgTs&&... args) : urng_ {std::forward<ArgTs>(args)...}
    {
    }

    template <typename FwdIterT>
        // requires(ForwardIterator(FwdIterT)
        //          && Assignable(FwdIterT::value_type, std::uint32_t))
    void
    generate(const FwdIterT first, const FwdIterT last)
    {
        std::uniform_int_distribution<std::uint32_t> dist {};
        std::generate(first, last, [this, &dist]() { return dist(urng_); });
    }
};
```

It would then be used like this.

```
std::seed_gen<std::random_device> generator {};
std::mt19937 engine {generator};
std::cout << engine() << '\n';
```

4.1.2 Integer Types

Another minor design variation would be to require `std::random_device::generate` to not assign 32 bit values to each element in the range but rather set all bits of the dereferenced iterator to independently chosen random values. This would be a more natural (and, in many contexts, more useful) behavior but it wouldn't be compatible with the existing specification of *seed sequence*. Therefore, it seems better to stay with the 32 bit requirement.

Conversely, `std::random_device::result_type` could be changed to `std::uint32_t` and its `operator()` be required to produce values in the range $[0, 2^{32})$. This would integrate better with the other facilities but could potentially break existing (brittle) code (on exotic platforms) in subtle ways. It could also affect performance adversely on platforms where `std::random_device::operator()` is implemented via a special hardware instruction and the result of that instruction is not a 32 bit value.

4.2 Shortcomings

A known minor shortcoming of the proposed solution is that the seed function (and corresponding constructor) would have to take their argument by non-const reference, so a temporary cannot bind to it which means that it is not possible to write the following code.

```
auto engine = std::mt19937 {std::random_device {}}; // won't compile
```

Instead, one has to write the slightly more verbose

```
std::random_device device {};  
auto engine = std::mt19937 {device};
```

which leaves the `std::random_device` in scope longer than necessary. Since it can be quite large and might hold an open file handle, this is potentially undesirable. To avoid this, a lambda can be used.

```
auto engine = [](){  
    std::random_device device {}; // Only lives as long as the lambda...  
    return std::mt19937 {device}; // ...and the copy is hopefully elided.  
}();
```

4.3 Impact on Standard Library Implementations

Little to no code should be required to change in the implementations of the random engines. A quick glance over the code of `libstdc++` [3] (GCC) and `libc++` [4] (Clang) has shown that there are no changes required for `libstdc++` and all that is needed for `libc++` is removing or weakening the concept check.

The to-be-added generate member function of `std::random_device` could be implemented in a compliant but naïve way like this.

```
template <typename RandIterT>  
    // requires(RandomAccessIterator(RandIterT)  
    //          && Unsigned(RandIterT::value_type)  
    //          && (std::numeric_limits<RandIterT::value_type>::digits >= 32))  
void  
random_device::generate(const RandIterT first, const RandIterT last)  
{  
    std::uniform_int_distribution<std::uint32_t> dist {};  
    std::generate(first, last, [this, &dist](){ return dist(*this); });  
}
```

Implementers will probably be interested to provide optimizations for the case that `RandIterT` is a pointer or can be converted to a pointer (such as `std::vector<unsigned>::iterator_type`) so that instead of calling `(*this)()` for each element, they can fill in all bytes at once if reading from a file like `/dev/urandom`. Care has to be taken to handle the case where `sizeof(RandIterT::value_type) > sizeof(std::uint32_t)` correctly. This enhanced `std::random_device` could be useful in other contexts, too.

It is worth mentioning that merely adding a member function does not break binary compatibility of existing code that uses `std::random_device`.

4.4 Implementations

A patch to implement this proposal for `libstdc++` can be found on the author's website [7] and will be continuously updated.

5. Proposed Wording

All proposed changes to the standard mentioned in this section are relative to N4140.

Add a new section before the existing § 26.5.1.2 [rand.req.seeq] to define *seed generator*.

Seed generator requirements [rand.req.seedgen]

A *seed generator* is an object that produces a requested number of unsigned integer values i with $0 \leq i < 2^{32}$. The generated numbers may be obtained from a non-deterministic source of entropy.

A class S satisfies the requirements of a seed generator if the expressions shown in the below table are valid

and have the indicated semantics and if S also satisfies all other requirements of this section. In that table and throughout this section:

- T is the type named by S's associated `result_type`;
- q is a value of S;
- rb and re are mutable random access iterators with an unsigned integer `value_type` of at least 32 bits.

Expression	Return type	Pre/post-condition	Complexity
<code>S::result_type</code>	T	T is an unsigned integer type of at least 32 bits.	compile-time
<code>q.generate(rb, re)</code>	void	Does nothing if <code>rb == re</code> . Otherwise, fills the supplied sequence [rb, re) with 32-bit quantities in a possibly non-deterministic way.	$O(\text{re} - \text{rb})$

In the existing section § 26.5.1.2 [rand.req.seedseq], change the beginning of the second paragraph as follows.

A class S satisfies the requirements of a seed sequence if it satisfies the requirements of a seed generator and in addition, the expressions [...]

In the current table 115, remove the first (`S::result_type`) and the fifth (`q.generate(rb, re)`) row which will already be covered by the *seed generator* requirements.

In section 26.5.6 [rand.device], add the following sentence at the end of the first paragraph.

It also meets the requirements of a seed generator.

Add the following declaration to the class overview of `std::random_device` right after the `operator()` under the “generating functions” section.

```
template <typename RandIterT> void generate(RandIterT first, RandIterT last);
```

After paragraph 7 of the same section, add the following specification.

```
template <typename RandIterT> void generate(RandIterT first, RandIterT last);
```

Requires: `RandIterT` must meet the requirements of random access iterator and its `value_type` must be an unsigned integer of at least 32 bits.

Effects: Assigns non-deterministic 32 bit random values uniformly distributed over the interval $[0, 2^{32})$ to the elements in the sequence [first, last). This function behaves as if it were implemented by the following code.

```
uniform_int_distribution<uint32_t> dist {};
generate(first, last, [this, &dist](){ return dist(*this); });
```

Throws: A value of an implementation-defined type derived from `exception` if a random number could not be obtained. It is unspecified in this case what and whether any values have been assigned to the elements in the range.

Replace section 26.5.1.4 [rand.req.eng] paragraph 4 letter d by the following sentence with the appropriate cross-reference to the section defining the requirements of *seed generator*.

q is an lvalue satisfying the requirements of a seed generator;

Re-structure the first seven rows of the current table 117 as follows. It is believed that this preserves the intended meaning of the current wording but avoids confusion.

Expression	Return type	Pre/post-condition	Complexity
<code>E()</code>		Creates an engine with the same initial state as all other default-constructed engines of type E.	$O(\text{size of state})$
<code>E(x)</code>		Creates an engine that compares equal to x.	$O(\text{size of state})$
<code>E(s)</code>		Creates an engine with an initial state as if by first default-constructing the engine and then calling <code>seed(s)</code> on it.	no worse than <code>E()</code> followed by <code>seed(s)</code>
<code>E(q)</code>		Creates an engine with an initial state as if by first default-constructing the engine and then calling <code>seed(q)</code> on it.	no worse than <code>E()</code> followed by <code>seed(q)</code>
<code>e.seed()</code>	void	Puts the engine into the same state as a default-constructed engine of type E.	$O(\text{size of state})$
<code>e.seed(s)</code>	void	Puts the engine into a state determined by s. [<i>Note:</i> For engines with an internal state larger than <code>sizeof(s)</code> , there will be impossible states after calling this function. — <i>end note</i>]	$O(\text{size of state})$
<code>e.seed(q)</code>	void	Puts the engine into a state that depends on a sequence produced by one call to <code>q.generate</code> for a range of the size of the engine's state.	same as complexity of <code>q.generate</code> called on a range of the size of the engine's state

Acknowledgments

Melissa O'Neill has written a series of remarkable blog posts that discuss the problems with seeding random engines in-depth. Although not the initial motivation for the author of this proposal, that blog post provided valuable theoretical and experimental support and the fact that its author had independently suggested basically the same addition to the standard library was very affirming.

The discussions with the principal author of `<random>`, Walter Brown, were very enlightening and helped the author a lot figuring out the final details of the proposal.

Nicol Bolas, Zhihao Yuan and Seth Cantrell have provided valuable feedback on the `std-proposals@isocpp.org` mailing list.

Baum mit Augen's shared experience of struggling to properly seed a `std::mt19937` from a `std::random_device` and the following discussion with Deduplicator (out of which came an earlier version of the code for `seed_non_deterministically_2nd`) were part of the motivation for writing this proposal [6].

References

1. Melissa O'Neill, *C++ Seeding Surprises*. 2015-04-16, <http://www.pcg-random.org/posts/cpp-seeding-surprises.html>
2. Nicol Bolas, via `std-proposals@isocpp.org`, 2016-01-03, <https://groups.google.com/a/isocpp.org/d/msg/std-proposals/sF-P4VE2Z3Q/u24T-g-hEgAJ>
3. The libc++ C++ Standard Library, <http://libcxx.llvm.org/>
4. The GNU Standard C++ Library Version 3, <https://gcc.gnu.org/libstdc++/>
5. Walter Brown, *Three <random> related Proposals*. N3547, 2013-03-12, <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2013/n3547.pdf>
6. “Seed `std::mt19937` from `std::random_device`” in: *Code Review*, 2015-10-30, <http://codereview.stackexchange.com/q/109260>
7. <http://klammler.eu/data/computer-science/iso-c++/p0205/>